**Università degli Studi di Messina**
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

# HANDS-ON 4:
Server TCP ad Alte Prestazioni con `epoll()`

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

# Indice

**1 Problem Statement** **2**

**2 Stato dell'Arte** **2**

**3 Metodologia ed Implementazione** **2**
3.1 Inizializzazione e Registrazione . . . . . 2
3.2 Event Loop Asincrono . . . . . 3

**4 Risultati** **3**
4.1 Test di Concorrenza . . . . . 3
4.2 Analisi Wireshark . . . . . 3
4.3 Performance Benchmark . . . . . 3

**5 Conclusioni e sviluppi futuri** **3**

# 1 Problem Statement

L'ottavo Hands-On (HO4) si focalizza sullo sviluppo di un server TCP ad alte prestazioni su piattaforma Linux. L'obiettivo primario è migrare dal modello di multiplexing tradizionale (`select`) alla moderna interfaccia **`epoll`**. Il sistema deve essere in grado di gestire simultaneamente fino a 10 connessioni client, garantendo l'integrità dei messaggi ricevuti e fornendo un feedback immediato (`ACK`$_E POLL_O K).Lasfidatecnicaconsistenell'ottimizzarel'usodellerisors$

# 2 Stato dell'Arte

Il kernel Linux ha introdotto epoll (Event Poll) per superare i limiti di scalabilità della select() e della poll().

- **Scalabilità O(1):** A differenza della select, la cui complessità cresce proporzionalmente al numero di FD monitorati ($O(N)$), epoll ha una complessità di risveglio costante $O(k)$, dove $k$ è il numero di eventi attivi.
- **Meccanismo Kernel-Side:** epoll mantiene una struttura dati ad albero (*Red-Black Tree*) nel kernel per tracciare i socket di interesse. Questo evita di copiare l'intero set di descrittori dallo spazio utente a quello kernel a ogni chiamata.
- **Trigger Modes:**
  - *Level Triggered (LT)*: Notifica l'evento finché il buffer non è vuoto.
  - *Edge Triggered (ET)*: Notifica l'evento solo al momento della transizione (nuovo dato). Richiede l'uso di socket non-bloccanti per evitare stalli.

# 3 Metodologia ed Implementazione

L'implementazione sfrutta le tre primitive cardine dell'API epoll per gestire il ciclo di vita delle connessioni.

## 3.1 Inizializzazione e Registrazione

Viene creata un'istanza di epoll e il socket di ascolto viene aggiunto alla lista di interesse:

**Snippet Registrazione epoll**

```
epoll_fd = epoll_create1(0);
ev.events = EPOLLIN; // Monitoraggio in lettura
ev.data.fd = listen_fd;
epoll_ctl(epoll_fd, EPOLL_CTL_ADD, listen_fd, &ev);
```

### 3.2 Event Loop Asincrono

`Il server rimane in attesa bloccante su epoll`$_w ait, cherestituiscesoloidescrittoripronti:$

# 4 Risultati

La validazione è stata condotta tramite test di carico sintetico e analisi comparativa.

### 4.1 Test di Concorrenza

L'esecuzione di 5 client simultanei ha confermato la corretta gestione dei File Descriptor. Il server ha riscontrato latenze minime, visualizzando sui log l'interlacciamento degli ID dei socket (fd:5, fd:6, ecc.) senza mostrare segni di stallo.

### 4.2 Analisi Wireshark

L'ispezione della cattura (Figura **??**) evidenzia che il server `epoll` risponde istantaneamente a ogni pacchetto `PSH` ricevuto. La stabilità della connessione è garantita dalla gestione asincrona degli acknowledgment a livello applicativo.

### 4.3 Performance Benchmark

I grafici prestazionali (Figura 4) mostrano che per carichi piccoli la differenza con la `select` è trascurabile, ma la stabilità del tempo di risposta conferma l'efficienza del meccanismo di notifica kernel-based.

# 5 Conclusioni e sviluppi futuri

L'Hands-On 4 ha dimostrato la superiorità tecnologica di `epoll` per la creazione di servizi di rete moderni. La riduzione della copia dei dati tra user-space e kernel rende questa soluzione lo standard per architetture cloud ad alte prestazioni.
**Sviluppi futuri:**

- **io**$_u ring : Esplorazionedelnuovosottosistema Linux$